

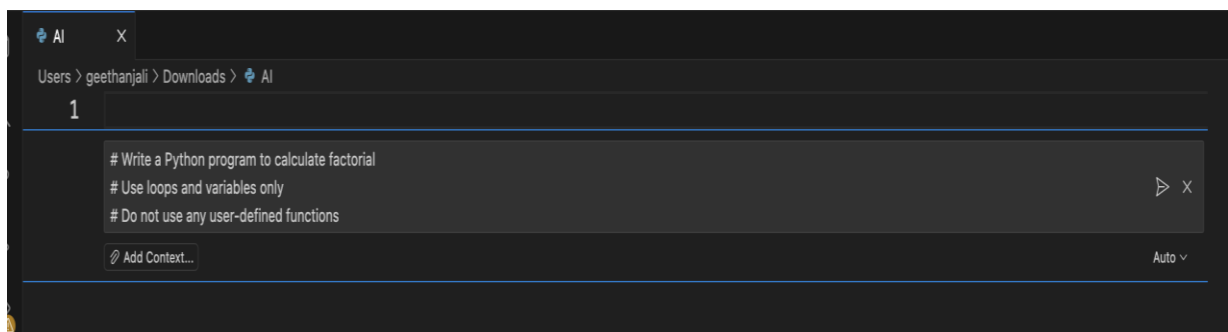
AI Assisted Coding

B.GEETHANJALI
(2303A51005)
BATCH-01

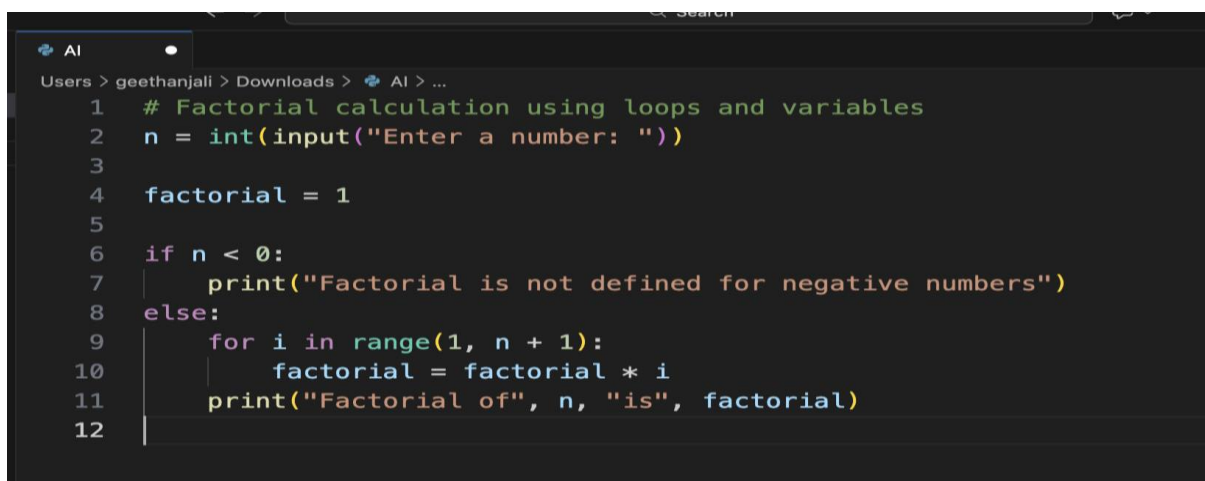
Week1 – Monday

Task 1: AI-Generated Logic Without Modularization (Factorial without Functions)

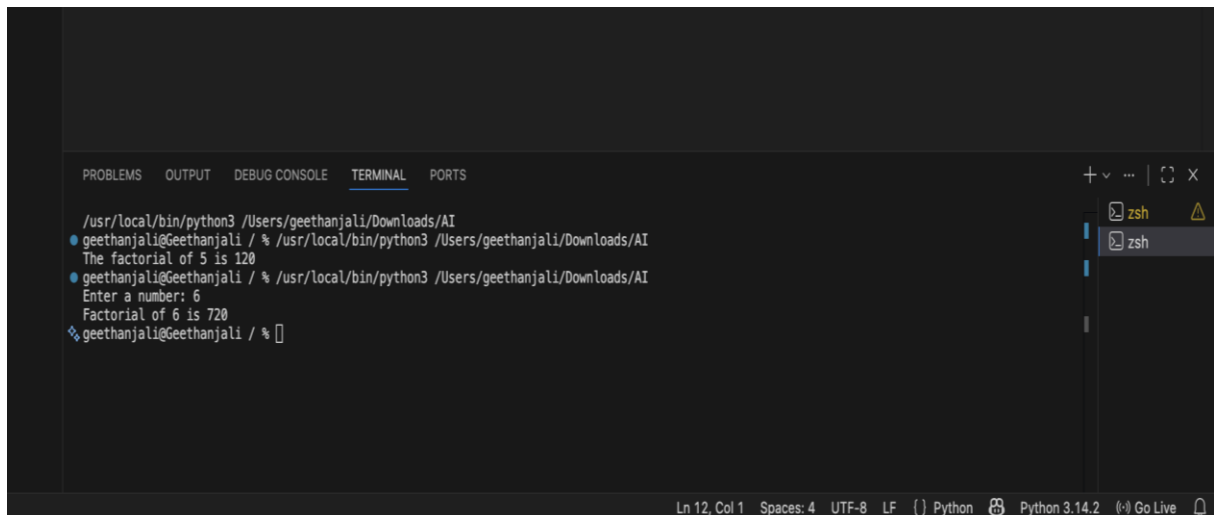
Prompt:



Copilot's suggestion:



Input/Output:



```

/usr/local/bin/python3 /Users/geethanjali/Downloads/AI
geethanjali@geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
The factorial of 5 is 120
geethanjali@geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Enter a number: 6
Factorial of 6 is 720
geethanjali@geethanjali / %

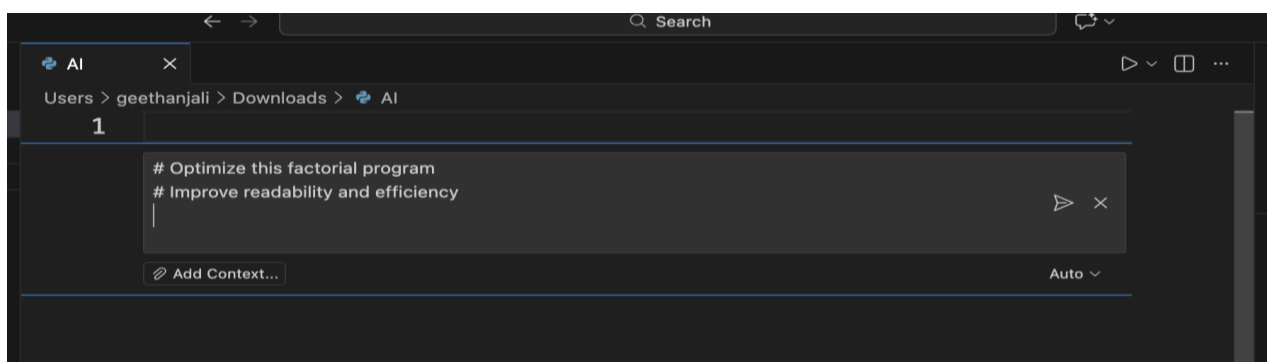
```

Brief reflection:

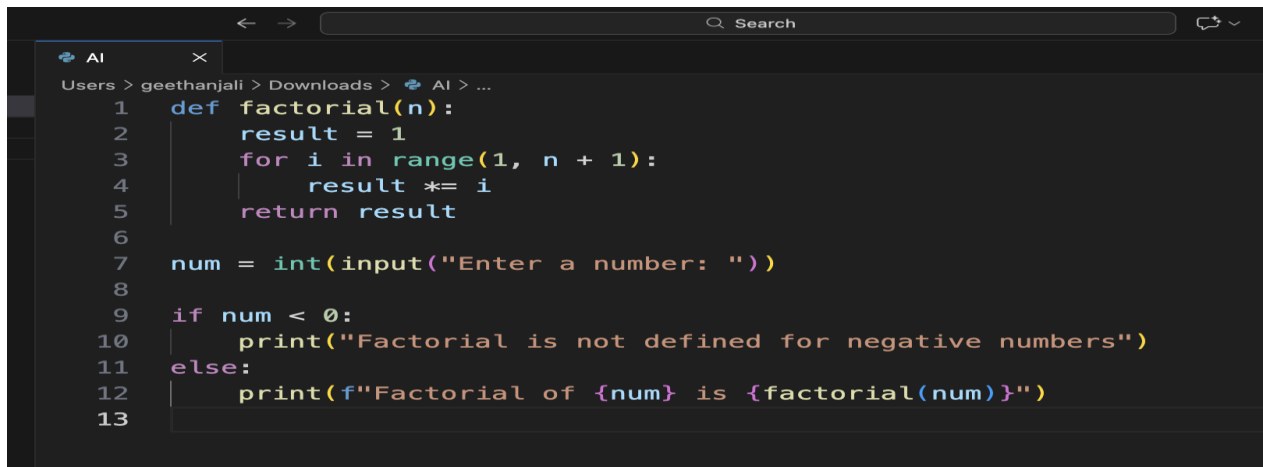
GitHub Copilot helped me write the program easily by suggesting code based on my comments. It made coding faster and less confusing for a beginner. The suggested code was correct and easy to understand. It also followed proper indentation and variable naming. However, I had to verify the logic and constraints myself. Copilot is helpful, but we should not depend on it completely

Task 2: AI Code Optimization & Cleanup (Improving Efficiency)

Prompt:

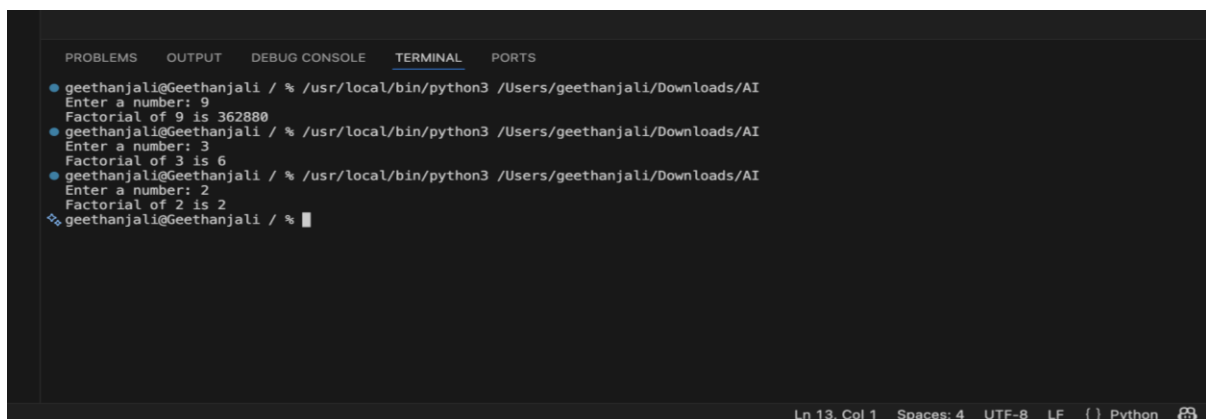


Copilot's suggestion:



```
1 def factorial(n):
2     result = 1
3     for i in range(1, n + 1):
4         result *= i
5     return result
6
7 num = int(input("Enter a number: "))
8
9 if num < 0:
10     print("Factorial is not defined for negative numbers")
11 else:
12     print(f"Factorial of {num} is {factorial(num)}")
13
```

Input/Output:



```
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Enter a number: 9
Factorial of 9 is 362880
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Enter a number: 3
Factorial of 3 is 6
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Enter a number: 2
Factorial of 2 is 2
geethanjali@Geethanjali / %
```

Explanation: What was improved?

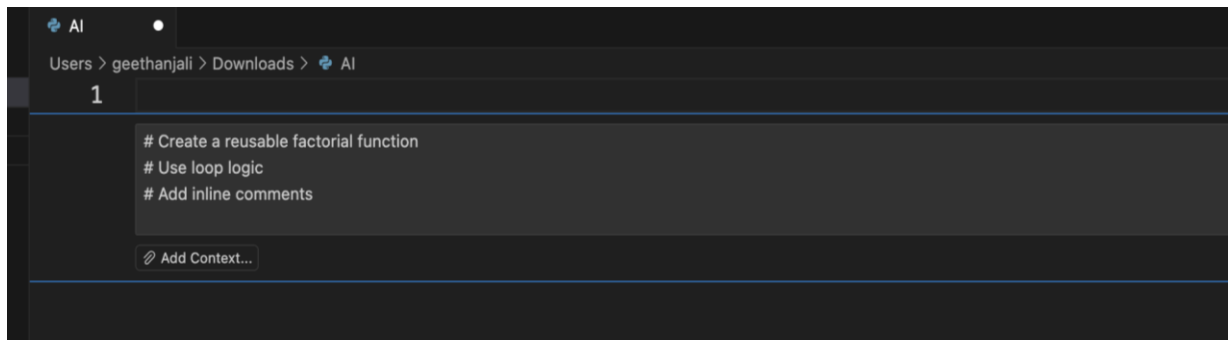
The code was cleaned up by simplifying the loop and reducing extra steps. Variable names and calculations were made clearer.

Why the new version is better?

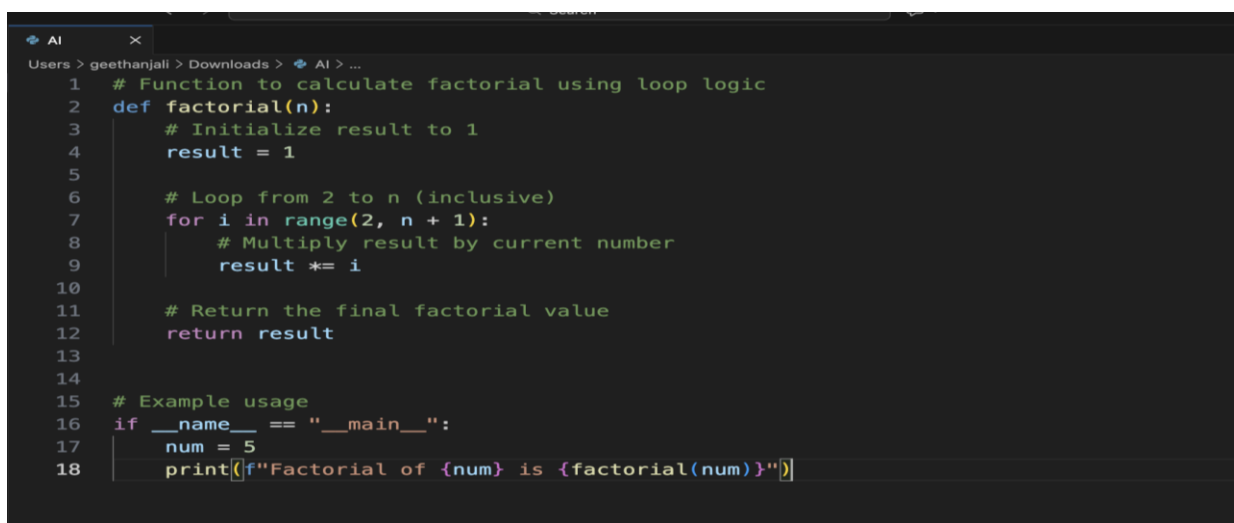
The improved code is easier to read and understand. It is cleaner, easier to maintain, and follows good coding practices, making it better for teamwork and future updates.

Task 3: Modular Design Using AI Assistance (Factorial with Functions)

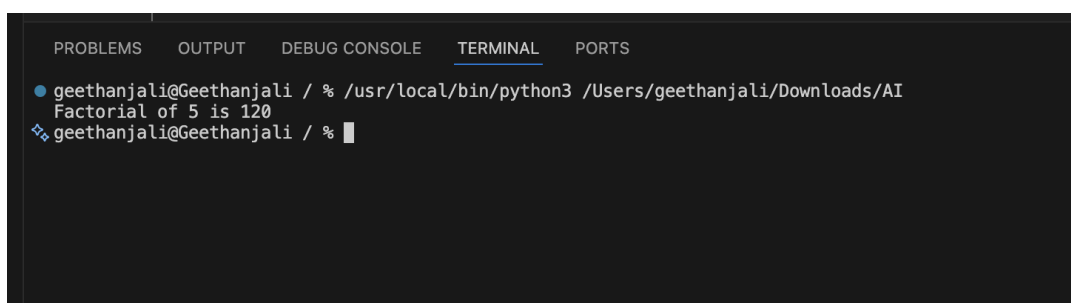
Prompt:



Copilot's suggestion:



Input/Output:



Short note: How modularity improves reusability

Using modular design makes the code reusable because the logic is written once inside a function and can be called multiple times in the same program or in other programs. This saves time and effort since we don't need to rewrite the same code. It also makes the program easier to understand, test, and maintain. If we need to make changes, we only update the function instead of modifying multiple copies of the

same logic. Overall, modularity improves efficiency, reduces errors, and helps when working on team projects.

Task 4: Comparative Analysis – Procedural vs Modular AI Code (With vs Without Functions)

A short technical report (300–400 words).

When comparing procedural and modular approaches for the factorial program generated by GitHub Copilot, several differences become clear.

In procedural code, the logic for calculating factorial is written directly in the main block along with input and output statements. While this works for small tasks, it mixes computation with user interaction, making it harder to read and follow. Reusing this logic in other scripts requires copying the same code multiple times, which increases the risk of errors and redundancy. Debugging is also more difficult because any issue requires examining the entire block of code. Procedural code is therefore not well-suited for larger projects or collaborative environments.

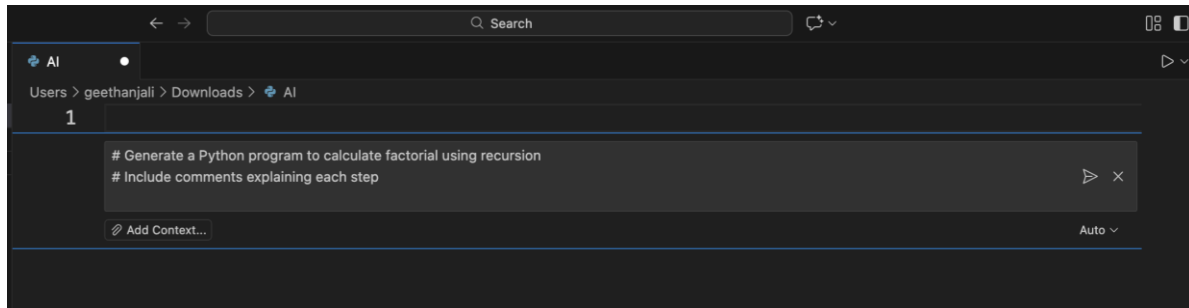
In contrast, the modular, function-based version separates the calculation logic into a dedicated function. This separation improves logic clarity, as the main program only handles input/output while the function handles computation. Reusability is a major advantage: the same function can be called multiple times within the same program or in other programs without rewriting the code. Debugging is easier because errors can be isolated and tested in the function alone. The modular approach also scales well for larger projects, allowing multiple team members to work on different parts of the code independently. Furthermore, reliance on AI is reduced, as the function can be reused across scripts without generating new AI suggestions each time.

Overall, while procedural code may be simpler for beginners or very small tasks, the modular function-based design is more professional, efficient, and maintainable. It improves readability, reduces redundancy, and supports scalable development. For teams and real-world projects, modular design clearly offers superior advantages.

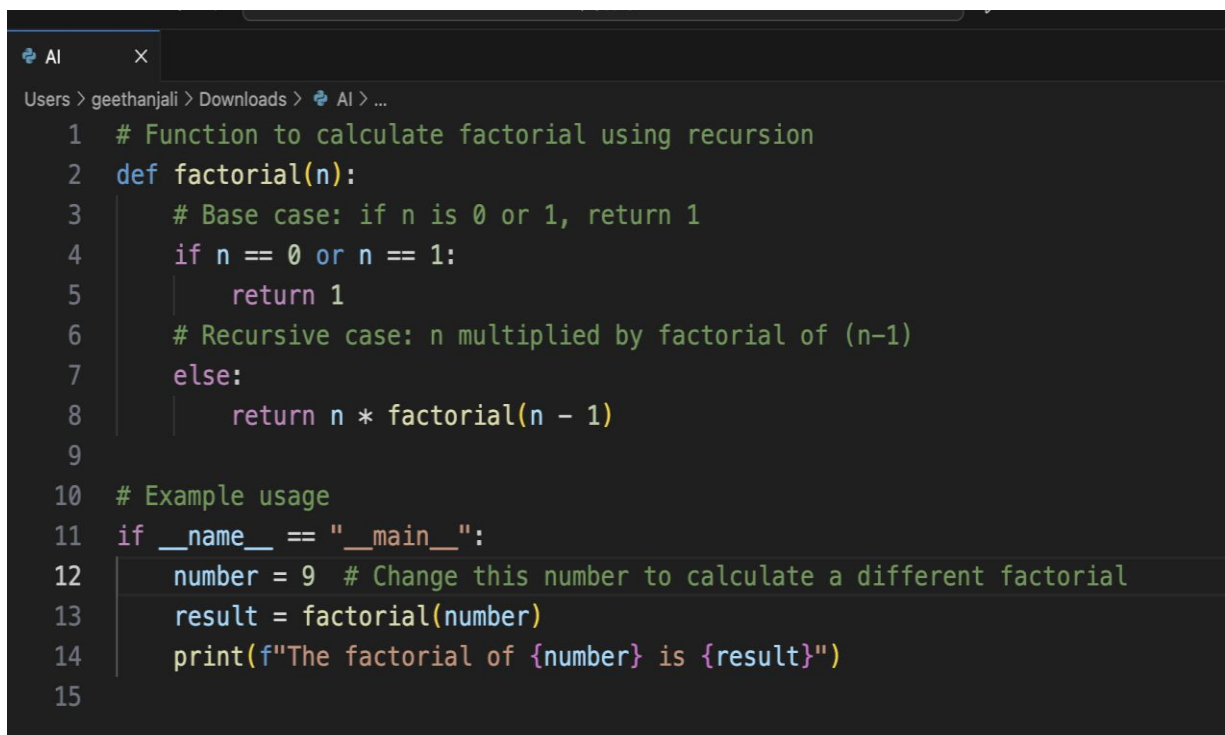
Task 5: AI-Generated Iterative vs Recursive Thinking

An iterative version of the logic

Prompt:



Copilot's suggestion:



Input/Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
The factorial of 9 is 362880
❖ geethanjali@Geethanjali / %
```

A recursive version of the same logic

Prompt:

```
AI
Users > geethanjali > Downloads > AI
1
# Convert the iterative factorial code into a recursive version
# Use a function
# Add clear inline comments
Add Context... Auto
```

Copilot's suggestion:

```
AI
Users > geethanjali > Downloads > AI > ...
1 # Recursive function to calculate factorial of a number
2 def factorial(n):
3     # Base case: factorial of 0 or 1 is 1
4     if n <= 1:
5         return 1
6
7     # Recursive case: multiply n by the factorial of (n-1)
8     return n * factorial(n - 1)
9
10
11 # Example usage
12 number = 12
13 result = factorial(number)
14 print(f"Factorial of {number} is {result}")
```

Input/Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
  Factorial of 12 is 479001600
🔧 geethanjali@Geethanjali / %
```

Comparison: Iterative vs Recursive Factorial

Criteria	Iterative Version	Recursive Version
1.Readability	Very easy to read; the logic flows linearly from start to end. Beginners can understand it quickly.	Slightly harder to read for beginners; understanding requires knowledge of recursion and function calls.
2.Stack Usage	Minimal; uses only variables in memory.	Uses the function call stack; each recursive call adds a new frame to the stack.
3.Performance Implications	Fast and memory-efficient; suitable for large inputs.	Slightly slower due to function call overhead; risk of stack overflow for very large inputs.
4.When Recursion is Not Recommended	N/A	Avoid recursion when the input number (n) is very large or when memory/stack size is limited, as this can cause stack overflow errors.